cuscope: CUDA Driver-API Interposer and GPU Memory Profiler

Oct 21, 2026 · @Adithya Sankar Menon

Problem statement

cuscope is a drop-in Linux library that records every GPU allocation, copy and kernel launch a CUDA program makes, without recompiling the program. You run `cuscope run -- ./app` and get a leak report, a memory and timing summary, and a timeline you can open in Perfetto.

When a CUDA program runs out of memory, leaks device memory or runs slower than expected, the developer usually can't see why from the outside. NVIDIA's own tools answer these questions using internal profiling interfaces. cuscope rebuilds a useful slice of that visibility from the outside, by sitting between the program and the driver.

It answers three questions:

1. Where did my GPU memory go? Live allocations, peak usage, leaks at exit, and frees of pointers that were never allocated.
2. Which kernels take the time? Per-kernel GPU durations, per stream, on a timeline.
3. Are my copies fast enough? Bytes, direction and effective bandwidth for each host-device copy.

Why the driver layer. Every CUDA program reaches the GPU through the driver library, `libcuda.so.1`, loaded dynamically. That holds even when the CUDA runtime is linked statically, which is `nvcc`'s default, and for frameworks like PyTorch. The catch is that the runtime does not call driver functions by normal symbol lookup. It loads `libcuda` with `dlopen` and fetches each function through `cuGetProcAddress`, asking for a base name (such as `cuMemAlloc`) plus a CUDA version, and the driver hands back the matching versioned function (NVIDIA: Driver Entry Point Access). Intercepting that lookup correctly is the core engineering problem of this project.

Goals, non-goals and success criteria

Phase 1 ships a profiler that works on unmodified binaries and reports numbers you have checked against a second source.

Goals (Phase 1)

- Intercept driver calls from all three common setups: a program built with `nvcc`'s default static runtime, one built with `-cudart shared`, and PyTorch.
- Track device, managed, pinned-host and stream-ordered (async pool) allocations, with call-site stack traces.
- Time every kernel launch and copy on the GPU, per stream.
- Export a Perfetto-compatible timeline and print a text summary at exit.
- Publish measured overhead and validation results in the README.

Non-goals

- Per-tensor tracking inside PyTorch. Its caching allocator suballocates large blocks, so cuscope sees segments, not tensors.
- Per-node timing inside CUDA graphs. A graph launch is recorded as one event.
- The virtual memory management APIs (`cuMemCreate`, `cuMemMap`).
- Multi-GPU, native Windows, and replacing Nsight.

Success criteria

Criterion	Target	How it's checked
Static and shared runtime builds	Identical reports for the same program	`static_vs_shared` test
Program behaviour	Output and return codes unchanged	Every test run with and without cuscope, outputs diffed
Leak detection	Exact count, sizes and call site	`leak.cu` test
Peak memory on PyTorch	Matches `torch.cuda.max_memory_reserved()`, every gap explained	Validation script
Overhead, allocation-only mode	Under 5% on a PyTorch training step (target, to be measured)	Benchmark suite
Overhead, full mode	Measured and published per call type	Microbenchmarks

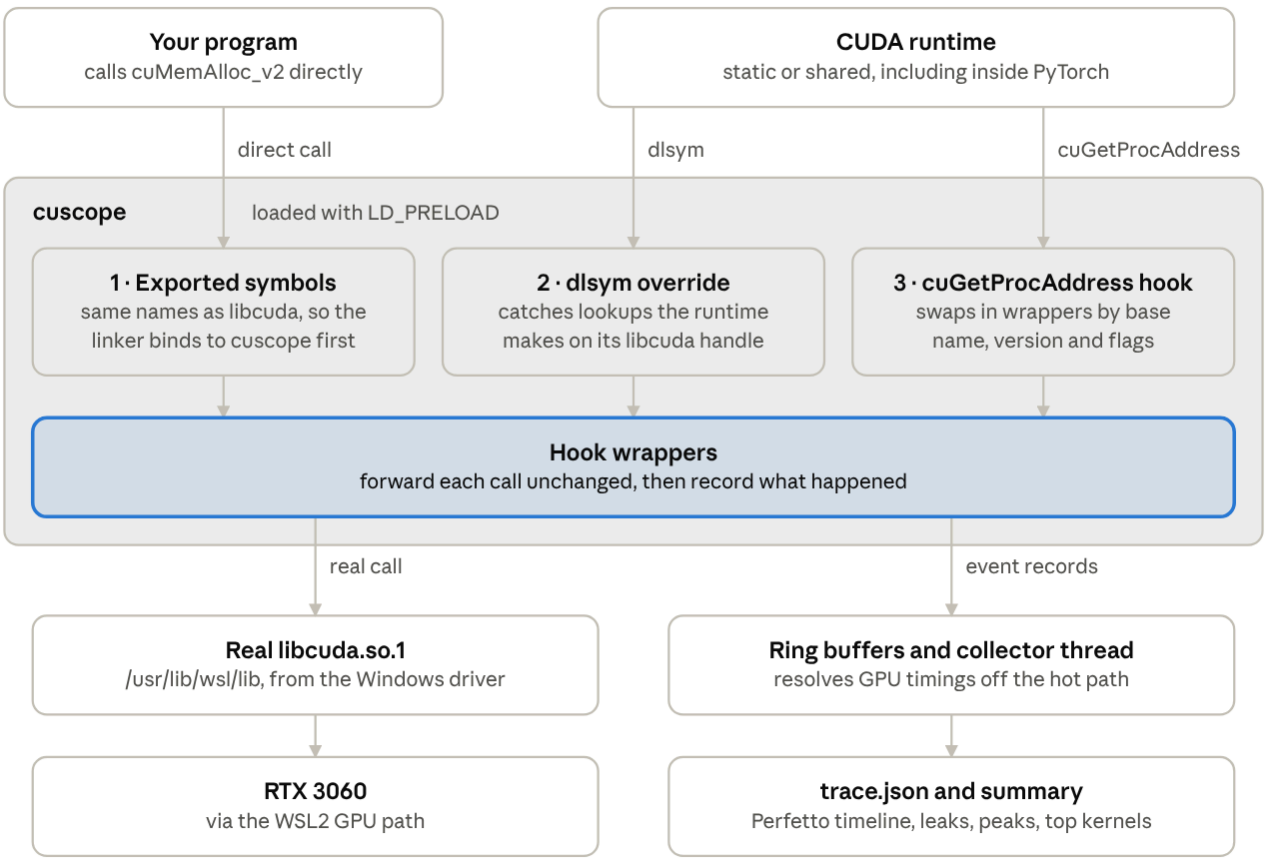
Environment and constraints

Development happens in WSL2 Ubuntu on a laptop RTX 3060 (Ampere, compute capability 8.6), using only user-space techniques.

- **Driver:** the Windows NVIDIA driver provides `libcuda.so.1` inside WSL2 under `/usr/lib/wsl/lib`. Never install a Linux NVIDIA driver inside WSL2; it breaks this setup.
- **Toolkit:** install the CUDA toolkit for WSL-Ubuntu inside WSL2. Pin one version and record it in the README, because driver function versions differ between CUDA releases.
- **Memory:** check VRAM with `nvidia-smi`; most laptop 3060s have 6 GB, which sizes the PyTorch workloads.
- **Language and build:** C++17 core library, CMake, Python 3 for the CLI and test harness.
- **No link-time dependency on libcuda.** cuscope compiles against `cuda.h` headers only and finds every real driver function at run time. That is what lets it load into any program, and lets CI build it on machines without a GPU.
- **What WSL2 rules out:** kernel modules and eBPF tracing of the driver. Nothing in Phase 1 needs them.

Architecture: three ways into the driver

cuscope is loaded with `LD_PRELOAD` and must catch driver calls on all three paths a program can use to reach `libcuda`. Missing any one of them means silently missing events.



cuscope architecture · 3 entry paths, 1 wrapper layer

Paths 1 and 2 hand over control by symbol name; path 3 hands over function pointers, so cuscope must keep the real pointer behind every wrapper it returns.

1. **Direct symbol binding.** A program linked with `-lcuda` calls `cuMemAlloc_v2` by name. cuscope exports a function with the same name, and because preloaded libraries are searched first, the dynamic linker binds the call to cuscope.
2. **`dlsym` lookup.** The CUDA runtime calls `dlopen("libcuda.so.1")` and then `dlsym(handle, ...)` to fetch `cuGetProcAddress`. A lookup on a specific handle searches only that library, so `LD_PRELOAD` alone never sees it. cuscope overrides `dlsym` itself: names in its hook table get cuscope's wrapper, everything else goes to the real `dlsym`.
3. **`cuGetProcAddress` lookup.** The runtime then asks the driver for each function by base name, CUDA version and flags. cuscope's wrapper calls the real `cuGetProcAddress` first. If the base name is in the hook table and the version matches the one wrapped, it stores the real pointer and returns the wrapper instead. Requests for `cuGetProcAddress` itself also get the wrapper.

Two details matter on path 3:

- **Per-thread default stream.** When the per-thread-default-stream flag is set, the driver returns a different variant of functions like `cuMemcpyAsync`. cuscope stores real pointers keyed by name *and* flags, so each wrapper forwards to the variant the program actually asked for.
- **Version mismatch.** If a program asks for an older version than the one cuscope wraps, cuscope returns the real pointer untouched. A wrapper with the wrong signature would corrupt arguments.

Getting the real functions. At first use, cuscope finds the real `dlsym` with `dlvsym` (glibc version `GLIBC_2.34`, falling back to `GLIBC_2.2.5`), opens `libcuda.so.1` itself, and resolves every real function through the real `cuGetProcAddress`. One resolution route for all three paths keeps the wrappers simple.

Components

Each component has one job and a narrow interface, so the allocation tracker and trace writer can be unit-tested without a GPU.

Component	Job	Interface and dependencies
Loader	Finds the real `dlsym`, opens `libcuda.so.1`, resolves real functions; runs once on first use	`real<T>(name, version, flags)`; depends on glibc only
Hook table	One list of every hooked function: base name, version, wrapper	An X-macro file (`hooks.def`); adding a hook is one line plus a wrapper
Wrappers	Thin functions that time the call, forward it, and report what happened	Call the real function, then the tracker and recorder
Allocation tracker	Live allocations, peak bytes per kind, leaks, invalid frees	`on_alloc(ptr, bytes, kind, stack)`, `on_free(ptr)`; 16 mutex-guarded shards keyed by pointer hash
Stack capture	Call-site backtraces, deduplicated	`backtrace()` to a configurable depth, stored by hash; symbolised at exit with `dladdr` and `__cxa_demangle`
Event recorder	Moves events off the hot path	Per-thread fixed-size ring buffers of 64-byte records; counts drops when full

Component	Job	Interface and dependencies
GPU timer	GPU start and end times for kernels and copies	Pool of CUDA events recorded around each call; collector thread resolves them
Trace writer	Perfetto-compatible timeline	Chrome Trace Event JSON; pure C++, no CUDA
Reporter	Human-readable summary at exit	Top kernels by time, peak memory, leaks with stacks, copy bandwidth
CLI	`cuscope run -- <cmd>`, `cuscope summary trace.json`	Python; sets `LD_PRELOAD` and config variables

Configuration uses environment variables: `CUSCOPE_OUT` (output path), `CUSCOPE_MODE` (`alloc` or `full`), `CUSCOPE_STACK_DEPTH`, and `CUSCOPE_SAMPLE_EVERY` (time every Nth launch).

Phase 1 hook list. The final list comes from the Day 2 census of what programs actually resolve; this is the starting point.

- Allocation: `cuMemAlloc`, `cuMemAllocPitch`, `cuMemFree`, `cuMemAllocAsync`, `cuMemAllocFromPoolAsync`, `cuMemFreeAsync`, `cuMemAllocManaged`, `cuMemAllocHost`, `cuMemHostAlloc`, `cuMemFreeHost`
- Copies: `cuMemcpy`, `cuMemcpyAsync`, `cuMemcpyHtoD`, `cuMemcpyDtoH`, `cuMemcpyDtoD`, and their `Async` variants
- Launches: `cuLaunchKernel`, `cuLaunchKernelEx`, `cuGraphLaunch` (one event per graph)
- Sync and teardown: `cuStreamSynchronize`, `cuCtxSynchronize`, `cuCtxDestroy`, `cuDevicePrimaryCtxRelease`, `cuDevicePrimaryCtxReset`
- Kernel names: `cuFuncGetName` where the driver provides it (NVIDIA: Execution Control), with a fallback map built by hooking `cuModuleGetFunction` and `cuLibraryGetKernel`

Data flow and trace format

The program's thread does only the minimum: forward the call, write one fixed-size record, return. Everything slow happens on a background collector thread or at exit.

On each hooked call (program thread)

1. Take a CPU timestamp (`clock_gettime(CLOCK_MONOTONIC)`).
2. For launches and copies in `full` mode: record a start event on the call's stream.

3. Call the real driver function with the same arguments.
4. Record the end event, update the tracker on success, and push a record into this thread's ring buffer.
5. Return the real function's result unchanged.

On the collector thread (every few milliseconds)

1. Drain every thread's ring buffer.
2. For pending GPU events, call `cuEventQuery` with the context the event was recorded in made current. Resolved events become GPU spans; unresolved ones wait.
3. Return finished events to the pool.

At exit or context teardown

1. Resolve or count every pending event before the context goes away.
2. Write the JSON trace and print the summary.

Aligning GPU and CPU clocks. On first use of each context, cuscope records a reference event, waits for it, and notes the CPU time. Every later GPU timestamp is that CPU time plus `cuEventElapsedTime(reference, event)`. Event timing resolution is around half a microsecond, which is fine for kernels in the microseconds-and-up range.

Record layout (64 bytes): type, thread id, CPU time, pointer, bytes, stream, function handle, correlation id, stack id. The correlation id ties a CPU-side launch to its GPU-side span.

How events appear in Perfetto

cuscope event	Trace Event JSON	Shown as
Driver API call	Complete event (`ph: X`) on the calling thread's track	CPU span per call
Kernel or copy on the GPU	Complete event on a per-stream track	GPU span per stream
Launch to execution	Flow events (`ph: s` and `ph: f`)	Arrow from CPU call to GPU span
Device memory in use	Counter event (`ph: C`)	Memory graph over time

Error handling and safety rules

The first rule overrides all others: cuscope must never change what the program does. A profiler that alters results is worse than no profiler.

- **Pass through exactly.** Every wrapper forwards the same arguments and returns the real function's result, including errors. cuscope records a failed allocation but never retries or masks it.
- **Fail open.** If initialisation fails or a real function can't be found, that hook becomes a plain pass-through and cuscope logs the problem once.
- **No recursion.** A thread-local depth flag marks calls cuscope makes itself (creating or recording events, capturing stacks), so they skip instrumentation.
- **Nothing slow on the hot path.** Ring buffers and the event pool are preallocated. `backtrace()` can allocate memory on its first call, so cuscope calls it once during initialisation to warm it up.
- **Full buffers drop, never block.** When a ring buffer is full, the record is dropped and counted. The summary prints the drop count, so a report never silently claims to be complete.
- **Teardown before exit.** The CUDA runtime's own exit cleanup can destroy the context before cuscope's exit handler runs. So cuscope resolves pending GPU events inside its context-teardown hooks, before calling the real teardown function.
- **Fork safety.** A `pthread_atfork` child handler resets cuscope's state and restarts the collector thread on next use.
- **Thread safety.** Tracker shards each have their own mutex; ring buffers have one writer and one reader each.
- **Output hygiene.** Logs go to `CUSCOPE_LOG` or stderr, never stdout, so the program's piped output is unchanged.

Testing and validation

Three layers: small CUDA programs with known answers, a cross-check against PyTorch's own numbers, and a GPU-free CI suite that runs on every push.

1. Known-answer test programs (run on your 3060; a Python harness parses the JSON report and asserts)

Test	What it does	Assertion
`static_vs_shared`	Same program built with default static runtime and with `-cudart shared`	Both reports identical. This is the test that proves the driver-level approach.

Test	What it does	Assertion
`leak.cu`	Three allocations, one freed	Two leaks, exact sizes, stack points at the right file and line
`double_free.cu`	Frees one pointer twice	One invalid free reported; program output unchanged
`async_pool.cu`	Stream-ordered alloc and free on two streams	Correct live bytes over time
`multistream.cu`	Kernels on four streams	Four GPU tracks, with overlap visible
`multithread.cpp`	Eight threads allocating and freeing at once	Exact counts, no crash
`bandwidth.cu`	Same-size copies from pinned and pageable memory	Both bandwidths reported; pinned is faster
Passthrough check	Every test with and without cuscope	stdout and exit code identical

2. Cross-check against PyTorch

- Run a small training loop (e.g. ResNet-18 on random data for 50 steps). cuscope's peak device memory should match `torch.cuda.max_memory_reserved()`; explain every gap, such as library allocations made outside PyTorch's allocator.
- Run once with `PYTORCH_NO_CUDA_MEMORY_CACHING=1` to show per-tensor allocations, and document the difference.
- If `torch.profiler` works on your WSL2 setup, compare kernel counts and total kernel time.

3. Overhead benchmarks

- Microbenchmarks: empty kernel launch, allocate-free pair, 4 KB copy. Report nanoseconds per call in three modes: off, `alloc`, `full`. Median of repeated runs.
- End to end: PyTorch training step time in each mode.

4. GPU-free CI (GitHub Actions)

- A mock `libcuda.so.1` implements about 15 driver functions on host memory.
- A test binary mimics the runtime: `dlopen`, `dlsym` for `cuGetProcAddress`, then lookups by base name and version. This exercises all three interception paths without a GPU.
- Unit tests for the tracker, stack deduplication and trace writer.

- AddressSanitizer and ThreadSanitizer builds of the mock suite.

Phase 1 plan: 14 days to v0.1.0

Phase 1 has one hard gate on Day 2 and ends with a tagged, documented release you can link in your application. Day 14 is buffer.

Day	Work	Done when
1	WSL2 setup: CUDA toolkit for WSL-Ubuntu, `nvidia-smi` and a sample program working. Repo skeleton with CMake, an empty preload library, a `hello.cu` test.	`LD_PRELOAD=./libcuscope.so ./hello` runs and prints a cuscope banner to stderr
2	**Symbol census.** Override `dlsym` and `cuGetProcAddress`; log every name, version and flag requested by the static build, shared build and PyTorch. Commit the logs.	**Gate:** census shows `cuMemAlloc` arriving through `cuGetProcAddress` in the static build. If not, switch to the proxy-library fallback (see Risks)
3	Loader and hook table: real `dlsym` via `dlvsym`, real functions via `cuGetProcAddress`, wrappers keyed by name and flags, recursion guard. All hooks pass through only.	All test programs run with identical output under cuscope
4	Allocation tracker and stack capture: every allocation kind, leak report at exit, invalid-free detection.	`leak.cu`, `double_free.cu` and `static_vs_shared` pass
5	Ring buffers, collector thread, trace writer: CPU API spans and memory counter track.	Trace opens in Perfetto with API calls and a memory graph
6	GPU timer: event pool, launch hooks, reference-event clock alignment.	Kernel spans appear on per-stream tracks
7	Kernel names (`cuFuncGetName` plus fallback map, demangled), flow arrows, teardown flush.	`multistream.cu` shows named kernels on four tracks with no unresolved events
8	Copy tracking: direction, bytes, GPU time, bandwidth.	`bandwidth.cu` reports pinned faster than pageable
9	CLI (`cuscope run`, `cuscope summary`) and the text summary report.	One command produces trace plus summary for any test

Day	Work	Done when
10	PyTorch cross-check; fix every discrepancy you can, document the rest.	Peak memory matches `max_memory_reserved()` or each gap is explained in writing
11	Overhead benchmarks; add `CUSCOPE_SAMPLE_EVERY` sampling if full mode is too slow.	Overhead table with real numbers
12	Mock `libcuda`, GitHub Actions CI, sanitizer builds.	CI green on push
13	README, `DESIGN.md` (this doc, trimmed), architecture diagram, Perfetto GIF, limitations, roadmap.	A stranger can install and run it from the README alone
14	Buffer. Fix what broke, tag `v0.1.0`, pin the repo, add the resume line.	Release tagged; repo linked in the application

If you fall behind, cut in this order: sampling mode (Day 11), then mock CI (Day 12), then flow arrows (Day 7). Never cut the PyTorch cross-check or the README; those are what make the project credible.

Phase 2: GPU sharing (weeks 3 and 4, after you apply)

Phase 2 turns the interception layer into a control layer: cuscope starts deciding whether a call goes through, not just watching it.

Week 3: per-process memory caps

- `CUSCOPE_MEM_LIMIT=2G` sets a cap. The allocation hooks keep an atomic byte count and return `CUDA_ERROR_OUT_OF_MEMORY` once a request would exceed it.
- The `cuMemGetInfo` hook reports the capped total and capped free memory, so PyTorch sizes its cache to the cap instead of the whole GPU.
- Limitation to document: context overhead and memory used inside the driver aren't visible to cuscope, so the cap is approximate.
- Demo: two PyTorch training jobs on the 3060, each capped, both finishing, compared with an uncapped run where one job runs out of memory.

Week 4: compute sharing

- A small daemon owns a shared-memory segment holding a token bucket per client process.

- The launch hook spends tokens equal to recently measured kernel time and blocks when the bucket is empty; the daemon refills buckets according to each process's configured share.
- Benchmark: throughput and latency per process at shares of 50/50 and 80/20, against no scheduler.
- Limitation to document: this throttles how fast kernels are submitted. It can't pause a kernel that is already running, so one long kernel still holds the GPU until it finishes.

If week 4 slips, it stays in the README roadmap; the memory caps alone are a complete, demoable feature.

Risks and fallbacks

The biggest risk is the Day 2 gate; everything else has a contained fallback.

Risk	Fallback
The `dlsym` override doesn't see the runtime's lookups under WSL2	Proxy library: a directory containing cuscope's own `libcuda.so.1`, placed first on `LD_LIBRARY_PATH`. It forwards every export to the real library (stubs generated from `nm -D` of the real one) and wraps the hooked ones. More code, same wrappers.
Runtime exit cleanup destroys the context before pending GPU events resolve	Resolve inside teardown hooks; anything still pending is counted as unresolved in the summary
Function names or versions differ in the installed CUDA release (12.x vs 13.x)	The census shows exactly what is requested; the hook table is keyed by base name plus version. Support the one pinned toolkit and say so in the README
Event per launch costs too much	`alloc` mode skips GPU timing; `CUSCOPE_SAMPLE_EVERY` times every Nth launch
Kernel names unavailable for some launches	Fall back to the name map, then to the function's address
PyTorch's caching allocator hides tensor-level allocations	Documented as a non-goal; show the `PYTORCH_NO_CUDA_MEMORY_CACHING=1` run for comparison
Schedule slips	Follow the cut order in the Phase 1 plan

Repo layout and GitHub presentation

The repo should let a reviewer understand the hard part in two minutes and run the tool in five.

```
cuscope/
  src/
    loader.cpp        real dlsym, libcuda, cuGetProcAddress
    hooks.def         the hook table (X-macro)
    wrappers/         alloc.cpp, memcpy.cpp, launch.cpp, teardown.cpp
    tracker.cpp       allocation tracker
    stacks.cpp        backtrace capture and symbolisation
    recorder.cpp      ring buffers and collector thread
    gputimer.cpp      event pool and clock alignment
    trace_writer.cpp  Chrome Trace Event JSON
    report.cpp        text summary
  cli/cuscope.py
  tests/
    cuda/             leak.cu, double_free.cu, multistream.cu, ...
    mock/             mock libcuda.so.1 and runtime-style test binary
    harness.py        runs tests, asserts on JSON reports
  bench/              microbenchmarks and PyTorch workload
  docs/
    DESIGN.md
    census/           Day 2 symbol census logs
    img/              architecture diagram, Perfetto GIF
  .github/workflows/ci.yml
```

README, in this order

1. One sentence on what it does, then the Perfetto GIF.
2. Quickstart: three commands from clone to trace.
3. How it works: the three interception paths, with the diagram. This is the section an NVIDIA reviewer will read most closely.
4. Validation: the PyTorch cross-check results.
5. Overhead: the benchmark table.
6. Limitations, stated plainly.
7. Roadmap: Phase 2.

Resume line, once you have the numbers: Built cuscope, an LD_PRELOAD profiler that intercepts the CUDA driver API (including cuGetProcAddress lookups) to track GPU allocations, kernels and copies in unmodified programs; matched PyTorch's peak-memory figures and kept overhead to X% on a training step.

Sources

- NVIDIA CUDA Programming Guide: Driver Entry Point Access
- NVIDIA CUDA Driver API: Execution Control (cuFuncGetName)